TP4 — Transactions et validation dans une blockchain avec Python

Matière : Blockchain
Environnement : Python 3, VS Code, PyCharm ou Jupyter Notebook
Bibliothèques : hashlib et time (incluses dans Python)

Objectifs pédagogiques

À la fin de ce TP, vous serez capables de :

- Comprendre le fonctionnement d'une transaction dans une blockchain.
- Identifier les informations nécessaires pour représenter une transaction.
- Comprendre la notion de solde et les conditions de validation d'une transaction.
- Implémenter une classe `Transaction` en Python.
- Vérifier qu'un utilisateur possède suffisamment de fonds avant un transfert.
- Comprendre le rôle des transactions en attente.
- Regrouper plusieurs transactions dans un bloc.
- Intégrer la validation des transactions dans une blockchain simplifiée.

Partie I — Rappel-cours : comprendre les transactions dans une blockchain

1. Rappel : fonctionnement des blocs

Dans les TP précédents, nous avons construit une blockchain simplifiée en Python.

Nous avons étudié :

- La structure d'un bloc.
- Le calcul des hash avec SHA-256.
- Le chaînage des blocs.
- Le minage et le Proof of Work.
- La vérification de l'intégrité de la chaîne.

Jusqu'à présent, chaque bloc contenait une seule transaction représentée par une chaîne de caractères.

Exemple :

"Alice envoie 10 euros a Bob"

Cependant, dans une véritable blockchain, les transactions possèdent une structure plus précise.

Un bloc peut également contenir plusieurs transactions.

Dans ce TP, nous allons apprendre à représenter, vérifier et regrouper des transactions avant de les enregistrer dans un bloc.

2. Qu'est-ce qu'une transaction ?

Une transaction est une opération enregistrée dans une blockchain.

Dans une blockchain de paiement, elle peut représenter un transfert de valeur entre deux utilisateurs.

Exemple

Alice souhaite envoyer 20 unités à Bob.

Pour représenter cette opération, nous devons connaître :

- L'expéditeur : Alice.
- Le destinataire : Bob.
- Le montant : 20 unités.

Nous pouvons représenter cette transaction sous la forme :

Alice → Bob : 20 unités

Dans notre programme, nous utiliserons une classe Python pour stocker ces informations.

Exemple de plusieurs transactions

Expéditeur	Destinataire	Montant
Alice	Bob	20
Bob	Charlie	10
Charlie	David	5
David	Alice	2

Chaque ligne représente une transaction indépendante.

Remarque : dans ce TP, les montants sont exprimés en unités fictives. Nous ne manipulons pas de véritables cryptomonnaies.

3. Comment une transaction est-elle validée ?

Dans une blockchain, une transaction ne doit pas être acceptée automatiquement.

Le système doit vérifier qu'elle respecte les règles du protocole.

Prenons un exemple.

Alice possède 100 unités.

Elle souhaite envoyer 30 unités à Bob.

Avant d'accepter cette opération, le système doit vérifier que le transfert est autorisé.

Dans notre modèle simplifié, nous vérifierons principalement :

1. Que l'expéditeur existe.
2. Que le destinataire existe.
3. Que le montant est strictement positif.
4. Que l'expéditeur possède suffisamment de fonds.

Exemple 1 : transaction possible

Solde initial d'Alice : 100 unités.

Montant envoyé : 30 unités.

Après la transaction :

- Alice possède 70 unités.
- Bob reçoit 30 unités supplémentaires.

Exemple 2 : transaction impossible

Solde initial d'Alice : 100 unités.

Montant demandé : 150 unités.

Alice ne possède pas suffisamment de fonds.

La transaction doit donc être refusée.

Pourquoi ces vérifications sont-elles nécessaires ?

Sans validation, un utilisateur pourrait essayer de transférer plus d'unités qu'il n'en possède.

Cela compromettrait la cohérence du registre.

Dans les véritables blockchains, d'autres vérifications sont également nécessaires, notamment les signatures numériques, les règles de dépense, les frais et la prévention des doubles dépenses.

4. Comprendre la notion de solde

Le solde représente la quantité d'unités disponibles pour un utilisateur.

Dans notre programme, nous utiliserons un dictionnaire Python pour représenter les soldes.

Exemple :

```
soldes = {
    "Alice": 100,
    "Bob": 50,
    "Charlie": 30
}
```

Chaque clé correspond à un utilisateur.

Chaque valeur représente son solde.

Pour consulter le solde d'Alice :

```
print(soldes["Alice"])
```

Pour diminuer le solde d'Alice de 20 unités :

```
soldes["Alice"] -= 20
```

Pour augmenter le solde de Bob de 20 unités :

```
soldes["Bob"] += 20
```

Dans cet exemple, nous utilisons un modèle simplifié fondé sur des comptes.

Il ne reproduit pas exactement le modèle UTXO utilisé par Bitcoin, dans lequel les transactions consomment et créent des sorties de transaction.

5. Qu'est-ce qu'une transaction en attente ?

Lorsqu'un utilisateur demande un transfert, celui-ci n'est pas nécessairement ajouté immédiatement à la blockchain.

Dans de nombreuses blockchains, les transactions sont d'abord diffusées et conservées temporairement dans une zone de transactions en attente, souvent appelée **mempool**.

Un validateur ou un mineur peut ensuite sélectionner certaines transactions pour les inclure dans un nouveau bloc.

Exemple

Trois transactions sont proposées :

- Alice → Bob : 10 unités.
- Bob → Charlie : 5 unités.
- Charlie → David : 2 unités.

Elles sont placées dans une liste de transactions en attente.

Lorsqu'un nouveau bloc est préparé, le programme sélectionne les transactions qui respectent les règles de validation.

Les transactions acceptées peuvent ensuite être enregistrées dans ce bloc.

Différence importante

Transaction en attente : opération proposée, mais pas encore enregistrée dans un bloc accepté.

Transaction confirmée : opération incluse dans un bloc accepté par le réseau selon les règles du protocole.

Dans notre simulation, nous considérerons qu'une transaction est confirmée lorsqu'elle est ajoutée à notre blockchain locale.

6. Comprendre la double dépense

La double dépense est un problème fondamental des monnaies numériques.

Elle correspond à une tentative d'utiliser les mêmes fonds pour plusieurs paiements incompatibles.

Exemple

Alice possède 50 unités.

Elle propose deux transactions :

- Transaction 1 : Alice envoie 40 unités à Bob.
- Transaction 2 : Alice envoie 30 unités à Charlie.

Si les deux transactions étaient acceptées sans tenir compte du solde restant, Alice pourrait dépenser 70 unités alors qu'elle n'en possède que 50.

Un système de validation doit empêcher cette situation.

Dans notre programme, nous traiterons les transactions dans un ordre déterminé et mettrons à jour les soldes au fur et à mesure.

Cela permettra de vérifier chaque nouvelle transaction en tenant compte des opérations précédentes.

7. Pourquoi regrouper plusieurs transactions dans un bloc ?

Une blockchain ne crée généralement pas un bloc distinct pour chaque transaction.

Plusieurs transactions peuvent être regroupées dans un même bloc.

Exemple :

Bloc 1

- Alice → Bob : 10 unités.
- Bob → Charlie : 5 unités.
- Charlie → David : 2 unités.

Bloc 2

- David → Alice : 1 unité.
- Alice → Charlie : 4 unités.

Le regroupement des transactions permet de les traiter et de les enregistrer collectivement.

Dans notre programme, chaque bloc contiendra une liste de transactions.

Pour calculer le hash d'un bloc, nous utiliserons une représentation déterministe de ses informations.

Cela signifie qu'un même bloc doit produire la même empreinte lorsque son contenu ne change pas.

Partie II — TP

Exercice 1 — Représenter une transaction avec Python

Étape 1 : préparation

Créez un dossier nommé :

`TP4_Blockchain`

Dans ce dossier, créez un fichier :

`exercice1.py`

Étape 2 : créer la classe Transaction

Écrivez le programme suivant :

```
class Transaction:

    def __init__(self, expediteur, destinataire, montant):

        self.expediteur = expediteur
        self.destinataire = destinataire
        self.montant = montant

    def afficher(self):

        print("Expediteur :", self.expediteur)
        print("Destinataire :", self.destinataire)
        print("Montant :", self.montant)
```

Étape 3 : créer une transaction

Ajoutez :

```
transaction1 = Transaction(
    "Alice",
    "Bob",
    20
)

transaction1.afficher()
```

Travail demandé

1. Exécutez le programme.
2. Identifiez les attributs de la classe `Transaction`.
3. Créez une deuxième transaction : Bob envoie 10 unités à Charlie.
4. Créez une troisième transaction : Charlie envoie 5 unités à David.
5. Affichez les informations des trois transactions.

Questions

1. Quel est le rôle de la classe `Transaction` ?
2. Pourquoi utilisons-nous une classe plutôt qu'une simple chaîne de caractères ?
3. Quelle est l'utilité de la méthode `afficher()` ?
4. Quels attributs supplémentaires pourrait-on ajouter à une transaction réelle ?

Exercice 2 — Gérer les soldes des utilisateurs

Étape 1 : créer le fichier

Créez un fichier nommé :

`exercice2.py`

Étape 2 : initialiser les soldes

Écrivez :

```
soldes = {
    "Alice": 100,
    "Bob": 50,
    "Charlie": 30,
    "David": 20
}
```

Étape 3 : afficher les soldes

Ajoutez :

```
def afficher_soldes(soldes):

    for utilisateur, solde in soldes.items():

        print(utilisateur, ":", solde, "unites")


afficher_soldes(soldes)
```

Étape 4 : effectuer un transfert

Ajoutez :

```
soldes["Alice"] -= 20
soldes["Bob"] += 20

afficher_soldes(soldes)
```

Travail demandé

1. Exécutez le programme.
2. Observez les soldes avant et après le transfert.
3. Effectuez un transfert de 10 unités de Bob vers Charlie.

4. Effectuez un transfert de 5 unités de Charlie vers David.
5. Affichez les soldes après chaque opération.

Questions

1. Comment le solde de l'expéditeur est-il modifié ?
2. Comment le solde du destinataire est-il modifié ?
3. Quelle est la somme totale des soldes avant les transferts ?
4. Quelle est la somme totale des soldes après les transferts ?
5. Que se passerait-il si Alice essayait d'envoyer 200 unités sans vérification préalable ?

Exercice 3 — Valider une transaction

Nous allons maintenant développer une fonction permettant de vérifier qu'une transaction respecte les règles définies dans notre système.

Étape 1 : créer le fichier

Créez un fichier nommé :

`exercice3.py`

Reprenez la classe `Transaction` de l'exercice 1.

Ajoutez le dictionnaire des soldes :

```
soldes = {
    "Alice": 100,
    "Bob": 50,
    "Charlie": 30,
    "David": 20
}
```

Étape 2 : créer une fonction de validation

Ajoutez :

```
def valider_transaction(transaction, soldes):

    if transaction.expediteur not in soldes:
        return False

    if transaction.destinataire not in soldes:
        return False

    if type(transaction.montant) not in (int, float):
        return False

    if transaction.montant <= 0:
        return False

    if soldes[transaction.expediteur] < transaction.montant:
        return False
```

```
    return True
```

Étape 3 : tester la fonction

Ajoutez :

```
transaction1 = Transaction(
    "Alice",
    "Bob",
    30
)

transaction2 = Transaction(
    "Charlie",
    "David",
    100
)

print(valider_transaction(transaction1, soldes))

print(valider_transaction(transaction2, soldes))
```

Travail demandé

1. Exécutez le programme.
2. Testez une transaction dont le montant est égal à zéro.
3. Testez une transaction dont le montant est négatif.
4. Testez une transaction avec un expéditeur inconnu.
5. Testez une transaction avec un destinataire inconnu.
6. Testez une transaction dont le montant dépasse le solde disponible.

Questions

1. Pourquoi doit-on vérifier l'existence de l'expéditeur ?
2. Pourquoi doit-on vérifier que le montant est positif ?
3. Quelle condition permet de détecter un solde insuffisant ?
4. Que signifie la valeur `False` retournée par la fonction ?
5. Cette fonction permet-elle de vérifier que la transaction a réellement été autorisée par l'expéditeur ? Justifiez.

Exercice 4 — Exécuter une transaction valide

Nous allons compléter le programme précédent pour mettre à jour les soldes uniquement lorsqu'une transaction est valide.

Étape 1 : ajouter la fonction

Reprenez le fichier `exercice3.py`.

Ajoutez :

```
def executer_transaction(transaction, soldes):
    if valider_transaction(transaction, soldes):
        soldes[transaction.expediteur] -= transaction.montant
        soldes[transaction.destinataire] += transaction.montant
        return True
    return False
```

Étape 2 : tester les transferts

Ajoutez :

```
transaction1 = Transaction(
    "Alice",
    "Bob",
    30
)

transaction2 = Transaction(
    "Bob",
    "Charlie",
    20
)

print(executer_transaction(transaction1, soldes))

print(executer_transaction(transaction2, soldes))

print(soldes)
```

Travail demandé

1. Exécutez le programme.
2. Observez les soldes après les transactions.
3. Ajoutez une transaction de 15 unités de Charlie vers David.
4. Ajoutez une transaction de 200 unités de David vers Alice.
5. Affichez les soldes après chaque opération.
6. Vérifiez si la somme totale des soldes a changé.

Questions

1. Pourquoi la fonction appelle-t-elle `valider_transaction()` ?
2. À quel moment les soldes sont-ils modifiés ?
3. Que se passe-t-il lorsqu'une transaction est refusée ?
4. Pourquoi est-il important de valider une transaction avant de l'exécuter ?

5. Quel problème pourrait apparaître si plusieurs transactions étaient exécutées sans tenir compte des soldes mis à jour ?

Exercice 5 — Gérer les transactions en attente

Dans cet exercice, nous allons créer une liste de transactions en attente.

Étape 1 : créer le fichier

Créez un fichier nommé :

`exercice5.py`

Reprenez la classe `Transaction`, le dictionnaire `soldes` et les fonctions développées dans les exercices précédents.

Étape 2 : créer les transactions

Ajoutez :

```
transactions_en_attente = [
    Transaction("Alice", "Bob", 20),
    Transaction("Bob", "Charlie", 10),
    Transaction("Charlie", "David", 5),
    Transaction("David", "Alice", 100)
]
```

Étape 3 : traiter les transactions

Ajoutez :

```
transactions_valides = []
transactions_refusees = []

for transaction in transactions_en_attente:

    if executer_transaction(transaction, soldes):

        transactions_valides.append(transaction)

    else:

        transactions_refusees.append(transaction)

print("Transactions valides :", len(transactions_valides))

print("Transactions refusees :", len(transactions_refusees))

print("Soldes :", soldes)
```

Travail demandé

1. Exécutez le programme.
2. Identifiez les transactions acceptées.
3. Identifiez les transactions refusées.

4. Modifiez le montant de la dernière transaction.
5. Exécutez à nouveau le programme.
6. Ajoutez une nouvelle transaction de 10 unités d'Alice vers Charlie.

Questions

1. Quel est le rôle de la liste `transactions_en_attente` ?
2. Pourquoi utilisons-nous deux listes distinctes pour les transactions acceptées et refusées ?
3. Pourquoi l'ordre de traitement des transactions peut-il influencer leur validation ?
4. Une transaction en attente est-elle nécessairement valide ?
5. Quelle différence existe-t-il entre une transaction acceptée par notre fonction et une transaction confirmée dans une véritable blockchain ?

Partie III — Intégrer les transactions dans une blockchain

Exercice 6 — Créer un bloc contenant plusieurs transactions

Dans le TP3, chaque bloc contenait une seule transaction.

Nous allons maintenant modifier la classe `Bloc` afin qu'elle puisse contenir une liste de transactions.

Étape 1 : créer le fichier

Créez un fichier nommé :

`exercice6.py`

Ajoutez :

```
import hashlib
import json

class Bloc:

    def __init__(self, index, transactions, previous_hash):

        self.index = index
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce = 0

        self.hash = self.calculer_hash()

    def calculer_hash(self):

        contenu = {
            "index": self.index,
            "transactions": [
                {
```

```
                    "expediteur": t.expediteur,
                    "destinataire": t.destinataire,
                    "montant": t.montant
                }
                for t in self.transactions
            ],
            "previous_hash": self.previous_hash,
            "nonce": self.nonce
        }

        texte = json.dumps(
            contenu,
            sort_keys=True,
            separators=(",", ":")
        )

        return hashlib.sha256(
            texte.encode("utf-8")
        ).hexdigest()

    def miner_bloc(self, difficulte):

        cible = "0" * difficulte

        while not self.hash.startswith(cible):

            self.nonce += 1
            self.hash = self.calculer_hash()
```

Étape 2 : créer les transactions

Reprenez la classe `Transaction` de l'exercice 1 et placez-la avant la classe `Bloc`.

Ajoutez ensuite :

```
transactions = [
    Transaction("Alice", "Bob", 20),
    Transaction("Bob", "Charlie", 10),
    Transaction("Charlie", "David", 5)
]
```

Étape 3 : créer et miner le bloc

Ajoutez :

```
bloc1 = Bloc(
    1,
    transactions,
    "0"
)

bloc1.miner_bloc(3)

print("Bloc :", bloc1.index)
print("Nombre de transactions :", len(bloc1.transactions))
print("Nonce :", bloc1.nonce)
print("Hash :", bloc1.hash)
```

Travail demandé

1. Exécutez le programme.
2. Affichez toutes les transactions contenues dans le bloc.
3. Modifiez le montant de la première transaction avant de créer le bloc.
4. Exécutez à nouveau le programme.
5. Ajoutez une quatrième transaction.
6. Recalculez le hash et minez le nouveau bloc.

Questions

1. Pourquoi utilisons-nous une liste de transactions ?
2. Quel est le rôle de la bibliothèque `json` ?
3. Pourquoi devons-nous convertir les informations du bloc en texte avant de calculer le hash ?
4. Pourquoi le hash dépend-il de toutes les transactions ?
5. Que se passe-t-il si une transaction est modifiée après le minage ?
6. Pourquoi le nonce intervient-il dans le calcul du hash ?

Exercice 7 — Construire une blockchain avec plusieurs transactions

Reprenez le programme de l'exercice 6.

Étape 1 : créer un premier bloc

```
transactions1 = [
    Transaction("Alice", "Bob", 20),
    Transaction("Bob", "Charlie", 10)
]

bloc1 = Bloc(
    1,
    transactions1,
    "0"
)

bloc1.miner_bloc(3)
```

Étape 2 : créer un deuxième bloc

```
transactions2 = [
    Transaction("Charlie", "David", 5),
    Transaction("David", "Alice", 2)
]

bloc2 = Bloc(
    2,
    transactions2,
    bloc1.hash
```

```
)

bloc2.miner_bloc(3)
```

Étape 3 : construire la blockchain

```
blockchain = [bloc1, bloc2]

for bloc in blockchain:

    print("--------------------------")
    print("Bloc :", bloc.index)
    print("Hash precedent :", bloc.previous_hash)
    print("Hash actuel :", bloc.hash)
    print("Nonce :", bloc.nonce)

    for transaction in bloc.transactions:

        transaction.afficher()
```

Travail demandé

1. Exécutez le programme.
2. Vérifiez le chaînage des deux blocs.
3. Ajoutez un troisième bloc contenant deux nouvelles transactions.
4. Minez le troisième bloc.
5. Affichez les trois blocs et leurs transactions.
6. Modifiez une transaction du premier bloc après le minage.
7. Comparez le hash enregistré avec le hash recalculé.

Questions

1. Pourquoi devons-nous miner le premier bloc avant de construire le deuxième ?
2. Pourquoi le deuxième bloc contient-il le hash du premier ?
3. Comment les transactions influencent-elles le hash d'un bloc ?
4. Pourquoi une modification du premier bloc peut-elle affecter les blocs suivants ?
5. Notre programme vérifie-t-il automatiquement les soldes avant d'ajouter les transactions aux blocs ? Justifiez.

Partie IV — Étude de cas : la double dépense

Exercice 8 — Détecter une tentative de double dépense

Considérons les soldes suivants :

Utilisateur	Solde initial
Alice	50

Bob	20
Charlie	10

Alice propose les transactions suivantes :

- Transaction 1 : Alice envoie 40 unités à Bob.
- Transaction 2 : Alice envoie 30 unités à Charlie.

Travail demandé

1. Créez le dictionnaire des soldes.
2. Créez les deux transactions.
3. Utilisez la fonction `executer_transaction()` pour traiter les transactions dans l'ordre.
4. Affichez les soldes après chaque tentative.
5. Inversez l'ordre des transactions et recommencez avec les soldes initiaux.
6. Comparez les résultats obtenus.

Tableau à compléter

Ordre de traitement	Transaction 1	Transaction 2	Solde final d'Alice
40 unités puis 30 unités			
30 unités puis 40 unités			

Questions

1. Pourquoi les deux transactions ne peuvent-elles pas être acceptées ensemble ?
2. Pourquoi l'ordre de traitement est-il important ?
3. Comment notre fonction empêche-t-elle cette double dépense simplifiée ?
4. Pourquoi ce problème est-il particulièrement important dans une monnaie numérique ?
5. Dans une véritable blockchain, pourquoi faut-il également vérifier l'autorisation des transactions et parvenir à un accord sur leur ordre ?

Partie V — Questions de compréhension

Répondez individuellement aux questions suivantes.

1. Qu'est-ce qu'une transaction dans une blockchain ?
2. Quelles informations avons-nous utilisées pour représenter une transaction ?
3. Pourquoi doit-on valider une transaction avant de l'exécuter ?
4. Quel est le rôle du solde dans notre modèle ?
5. Qu'est-ce qu'une transaction en attente ?
6. Quelle est la différence entre une transaction en attente et une transaction confirmée ?
7. Pourquoi plusieurs transactions peuvent-elles être regroupées dans un même bloc ?

8. Comment les transactions interviennent-elles dans le calcul du hash ?
9. Qu'est-ce que la double dépense ?
10. Pourquoi l'ordre des transactions est-il important ?
11. Quelle différence existe-t-il entre la validation des transactions et le minage d'un bloc ?
12. Pourquoi notre simulation ne constitue-t-elle pas encore un véritable système de paiement sécurisé ?
13. Quel est le rôle des signatures numériques dans les véritables blockchains ?
14. Pourquoi un mécanisme de consensus reste-t-il nécessaire même lorsque toutes les transactions sont valides ?

Partie VI — Synthèse

Complétez le tableau suivant.

Notion	Définition ou explication	Exemple
Transaction		
Expéditeur		
Destinataire		
Solde		
Validation d'une transaction		
Transaction en attente		
Transaction confirmée		
Bloc contenant plusieurs transactions		
Double dépense		
Minage d'un bloc		
Intégrité des transactions		

Travail à rendre

Chaque étudiant doit remettre :

1. Les fichiers Python des exercices réalisés.
2. Le programme permettant de créer et de valider des transactions.
3. Le programme de gestion des soldes.
4. Le programme permettant de traiter les transactions en attente.
5. La blockchain contenant au moins trois blocs avec plusieurs transactions.
6. Le tableau complété de l'exercice sur la double dépense.
7. Les réponses aux questions des exercices et de compréhension.